BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 6

Roll no: 04 PRN No: 12412054 Name of Student: Narendra Ajit Patil

Subject Name : Deep Learning

Problem Statement :

Design and implement a Convolutional Neural Network (CNN) for image classification using the Tomato or Soybean disease dataset.

Algorithm :

1) Start

2) Load the Tomato Leaf Disease Dataset containing images belonging to multiple disease classes.

3) Identify the classes from the subfolder names.

4) Preprocess the images:
   - Resize each image to 128 × 128 pixels.
   - Convert images into numerical pixel arrays.
   - Normalize pixel values from 0–255 to 0–1.

5) Split the dataset into:
   - 80% Training data
   - 20% Validation data

6) Apply data augmentation to training images using horizontal flipping, rotation, and zooming.

7) Initialize the CNN architecture:
   - Input layer

- Convolutional layer with 32 filters
- Max-pooling layer
- Convolutional layer with 64 filters

- Max-pooling layer
- Convolutional layer with 128 filters
- Max-pooling layer
- Flatten layer
- Fully connected Dense layer with 128 neurons
- Dropout layer with 0.5
- Softmax output layer with 10 classes

7) Compile the CNN using:
   - Optimizer: Adam
   - Loss function: Sparse Categorical Crossentropy
   - Evaluation metric: Accuracy

8) Train the CNN using the training dataset for a specified number of epochs.

9) Validate the model after each epoch using the validation dataset.

10) Evaluate model performance using validation accuracy and validation loss.

11) Generate predictions for the validation images.

12) Generate a confusion matrix to compare actual and predicted disease classes.

13) Calculate precision, recall, and F1-score using the classification report.

14) Save the trained CNN model for future disease prediction.

15) Input a new tomato leaf image and preprocess it in the same way.

16) Pass the image through the trained CNN.

17) Predict the disease class having the highest Softmax probability.

18) Display the predicted disease and confidence score.

19) Stop.

Code :

```
import tensorflow as tf
print("TensorFlow version:", tf.__version__)
print("GPU:", tf.config.list_physical_devices('GPU'))
!pip install -q Kaggle
!kaggle datasets download -d zunorain/tomato-dataset-v2 -p /content
!ls -lh /content/*.zip
!unzip -t /content/tomato-dataset-v2.zip | tail -5
import zipfile
import os

zip_path = "/content/tomato-dataset-v2.zip"
extract_path = "/content/tomato_dataset"

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)

print("Dataset extracted successfully!")
import os

for root, dirs, files in os.walk("/content/tomato_dataset"):
    print(root)
    print("Folders:", dirs[:15])

    if root.count(os.sep) > 4:
        dirs[:] = []
import os

base_path = "/content/tomato_dataset"

print("Contents of dataset:")
for item in os.listdir(base_path):
    print(" -", item)

for root, dirs, files in os.walk(base_path):
    level = root.replace(base_path, "").count(os.sep)
```

```
    indent = "    " * level
    print(f"{indent}{os.path.basename(root)}/")

    if level < 2:
        for file in files[:5]:
            print(f"{indent}    {file}")
train_dir = None
test_dir = None

for root, dirs, files in os.walk(base_path):
    folder_names = [d.lower() for d in dirs]

    if "train" in folder_names:
        train_dir = os.path.join(root, dirs[folder_names.index("train")])

    if "test" in folder_names:
        test_dir = os.path.join(root, dirs[folder_names.index("test")])

print("Train directory:", train_dir)
print("Test directory:", test_dir)
import os

base_path = "/content/tomato_dataset"

print("Contents of /content/tomato_dataset:")
print(os.listdir(base_path))
print("\nFull folder structure:\n")

for root, dirs, files in os.walk(base_path):
    level = root.replace(base_path, "").count(os.sep)
    indent = "    " * level

    print(indent + os.path.basename(root) + "/")

    # Show first 3 files if present
    for file in files[:3]:
        print(indent + "    " + file)
```

```
    if level >= 3:
        dirs[:] = []
image_extensions = (".jpg", ".jpeg", ".png", ".bmp", ".webp")

print("\nFolders containing images:\n")

for root, dirs, files in os.walk(base_path):

    image_count = sum(
        1 for f in files
        if f.lower().endswith(image_extensions)
    )

    if image_count > 0:
        print(root, "→", image_count, "images")
import os

dataset_dir = "/content/tomato_dataset/augmented_dataset"

print("Dataset path:", dataset_dir)
print("\nClasses:")

classes = sorted([
    folder
    for folder in os.listdir(dataset_dir)
    if os.path.isdir(os.path.join(dataset_dir, folder))
])

for i, class_name in enumerate(classes):
    print(i, ":", class_name)

print("\nTotal classes:", len(classes))
image_extensions = (".jpg", ".jpeg", ".png", ".bmp", ".webp")

total_images = 0
```

```
print("Images per class:\n")

for class_name in classes:

    class_path = os.path.join(dataset_dir, class_name)

    count = sum(
        1
        for file in os.listdir(class_path)
        if file.lower().endswith(image_extensions)
    )

    print(f"{class_name}: {count}")

    total_images += count

print("\nTotal images:", total_images)
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from tensorflow.keras import layers, models
from sklearn.metrics import confusion_matrix, classification_report

print("TensorFlow version:", tf.__version__)
print("GPU:", tf.config.list_physical_devices("GPU"))
IMG_SIZE = (128, 128)
BATCH_SIZE = 32
SEED = 42
train_ds = tf.keras.utils.image_dataset_from_directory(
    dataset_dir,
    validation_split=0.2,
    subset="training",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
```

```
    shuffle=True
)
val_ds = tf.keras.utils.image_dataset_from_directory(
    dataset_dir,
    validation_split=0.2,
    subset="validation",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    shuffle=False
)
class_names = train_ds.class_names

print("Number of classes:", len(class_names))

for i, name in enumerate(class_names):
    print(i, ":", name)
plt.figure(figsize=(12, 9))

for images, labels in train_ds.take(1):

    for i in range(min(12, len(images))):

        ax = plt.subplot(3, 4, i + 1)

        plt.imshow(images[i].numpy().astype("uint8"))

        plt.title(class_names[labels[i]])

        plt.axis("off")

plt.tight_layout()
plt.show()
AUTOTUNE = tf.data.AUTOTUNE

train_ds = train_ds.prefetch(
    buffer_size=AUTOTUNE
```

```
)

val_ds = val_ds.prefetch(
    buffer_size=AUTOTUNE
)
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1)
])
num_classes = len(class_names)

model = models.Sequential([

    # Input
    layers.Input(shape=(128, 128, 3)),

    # Data augmentation
    data_augmentation,

    # Normalize pixels from 0-255 to 0-1
    layers.Rescaling(1./255),

    # Convolution Block 1
    layers.Conv2D(
        32,
        (3, 3),
        activation="relu"
    ),

    layers.MaxPooling2D(
        (2, 2)
    ),

    # Convolution Block 2
    layers.Conv2D(
        64,
```

```
        (3, 3),
        activation="relu"
    ),

    layers.MaxPooling2D(
        (2, 2)
    ),

    # Convolution Block 3
    layers.Conv2D(
        128,
        (3, 3),
        activation="relu"
    ),

    layers.MaxPooling2D(
        (2, 2)
    ),

    # Flatten
    layers.Flatten(),

    # Fully connected layer
    layers.Dense(
        128,
        activation="relu"
    ),

    # Dropout
    layers.Dropout(0.5),

    # Output
    layers.Dense(
        num_classes,
        activation="softmax"
    )
])
```

```
model.summary()
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
EPOCHS = 10

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS
)
plt.figure(figsize=(8, 5))

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)

plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.xlabel("Epoch")
plt.ylabel("Accuracy")

plt.title("Training vs Validation Accuracy")

plt.legend()
plt.grid()

plt.show()
plt.figure(figsize=(8, 5))

plt.plot(
```

```
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.title("Training vs Validation Loss")

plt.legend()
plt.grid()

plt.show()
loss, accuracy = model.evaluate(val_ds)

print("Validation Loss:", loss)
print("Validation Accuracy:", accuracy)
print("Validation Accuracy (%):", round(accuracy * 100, 2))
y_true = []
y_pred = []

for images, labels in val_ds:

    predictions = model.predict(
        images,
        verbose=0
    )

    predicted_labels = np.argmax(
        predictions,
        axis=1
    )
```

```
    y_true.extend(labels.numpy())
    y_pred.extend(predicted_labels)

y_true = np.array(y_true)
y_pred = np.array(y_pred)
cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(12, 10))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    xticklabels=class_names,
    yticklabels=class_names
)

plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.title("Confusion Matrix")

plt.xticks(rotation=45, ha="right")
plt.yticks(rotation=0)

plt.tight_layout()
plt.show()
from sklearn.metrics import classification_report

print(
    classification_report(
        y_true,
        y_pred,
        labels=list(range(len(class_names))),
        target_names=class_names,
        zero_division=0
    )
```

```
)
model.save("/content/tomato_disease_cnn.keras")

print("Model saved successfully!")
```

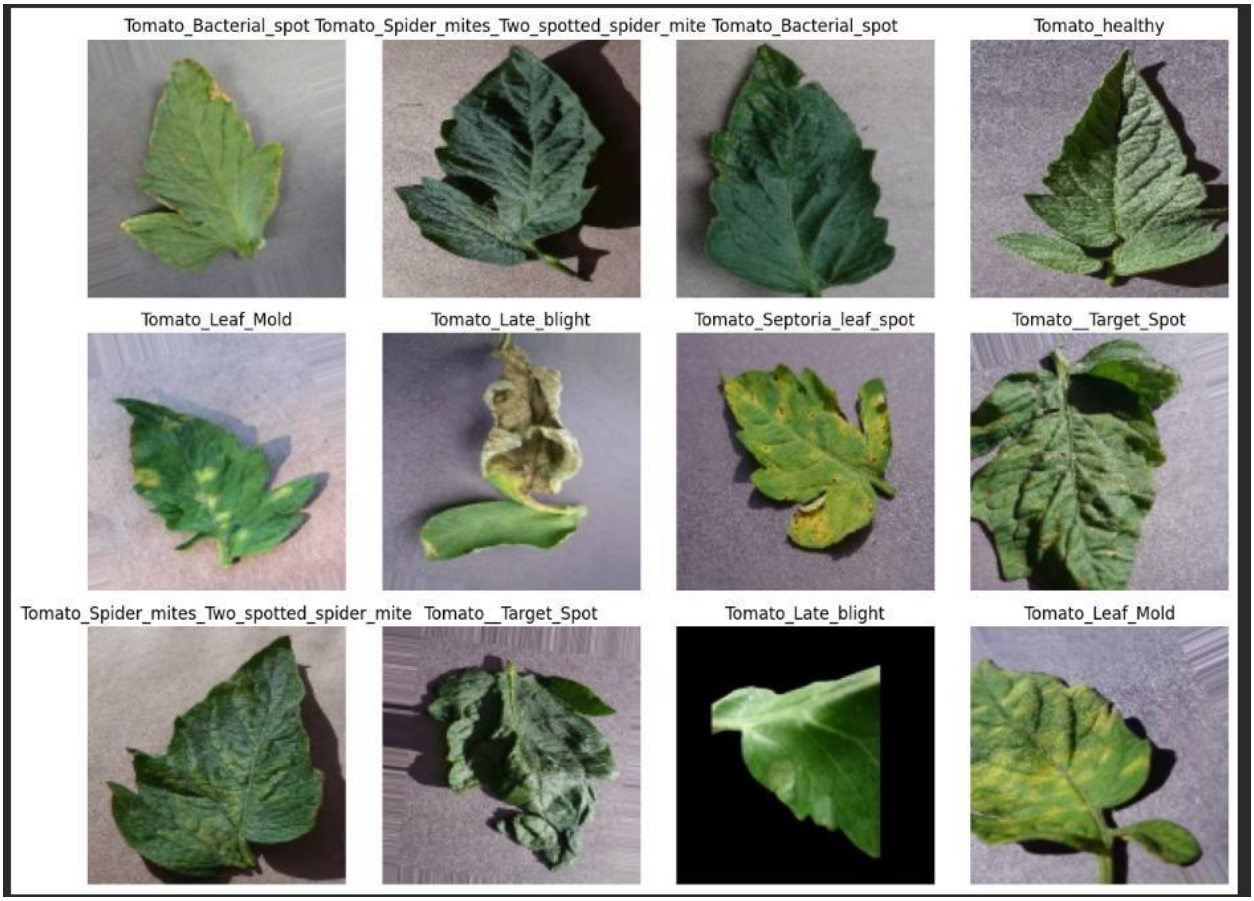
Output :

```
Full folder structure:

tomato_dataset/
    augmented_dataset/
        Tomato__Tomato_YellowLeaf__Curl_Virus/
            img_601.jpg
            img_210.jpg
            img_1363.jpg
        Tomato_Bacterial_spot/
            img_601.jpg
            img_210.jpg
            img_1363.jpg
        Tomato_Late_blight/
            img_601.jpg
            img_210.jpg
            img_1363.jpg
        Tomato_Leaf_Mold/
            img_601.jpg
            img_210.jpg
            img_1363.jpg
        Tomato_Spider_mites_Two_spotted_spider_mite/
            img_601.jpg
            img_210.jpg
            img_1363.jpg
        Tomato_Septoria_leaf_spot/
            img_601.jpg
            img_210.jpg
            img_1363.jpg
        Tomato__Target_Spot/
            img_601.jpg
            img_210.jpg
            img_1363.jpg
        Tomato_Early_blight/
            img_601.jpg
            img_210.jpg
            img_1363.jpg
        Tomato_healthy/
            img_601.jpg
            img_210.jpg
            img_1363.jpg
        Tomato__Tomato_mosaic_virus/
            img_601.jpg
            img_210.jpg
            img_1363.jpg
```

```
Folders containing images:

/content/tomato_dataset/augmented_dataset/Tomato__Tomato_YellowLeaf__Curl_Virus → 3208 images
/content/tomato_dataset/augmented_dataset/Tomato_Bacterial_spot → 3164 images
/content/tomato_dataset/augmented_dataset/Tomato_Late_blight → 3106 images
/content/tomato_dataset/augmented_dataset/Tomato_Leaf_Mold → 2975 images
/content/tomato_dataset/augmented_dataset/Tomato_Spider_mites_Two_spotted_spider_mite → 3104 images
/content/tomato_dataset/augmented_dataset/Tomato_Septoria_leaf_spot → 3112 images
/content/tomato_dataset/augmented_dataset/Tomato__Target_Spot → 3064 images
/content/tomato_dataset/augmented_dataset/Tomato_Early_blight → 2963 images
/content/tomato_dataset/augmented_dataset/Tomato_healthy → 3072 images
/content/tomato_dataset/augmented_dataset/Tomato__Tomato_mosaic_virus → 2841 images
```

```
Dataset path: /content/tomato_dataset/augmented_dataset

Classes:
0 : Tomato_Bacterial_spot
1 : Tomato_Early_blight
2 : Tomato_Late_blight
3 : Tomato_Leaf_Mold
4 : Tomato_Septoria_leaf_spot
5 : Tomato_Spider_mites_Two_spotted_spider_mite
6 : Tomato__Target_Spot
7 : Tomato__Tomato_YellowLeaf__Curl_Virus
8 : Tomato__Tomato_mosaic_virus
9 : Tomato_healthy

Total classes: 10
```

```
Images per class:

Tomato_Bacterial_spot: 3164
Tomato_Early_blight: 2963
Tomato_Late_blight: 3106
Tomato_Leaf_Mold: 2975
Tomato_Septoria_leaf_spot: 3112
Tomato_Spider_mites_Two_spotted_spider_mite: 3104
Tomato__Target_Spot: 3064
Tomato__Tomato_YellowLeaf__Curl_Virus: 3208
Tomato__Tomato_mosaic_virus: 2841
Tomato_healthy: 3072

Total images: 30609
```

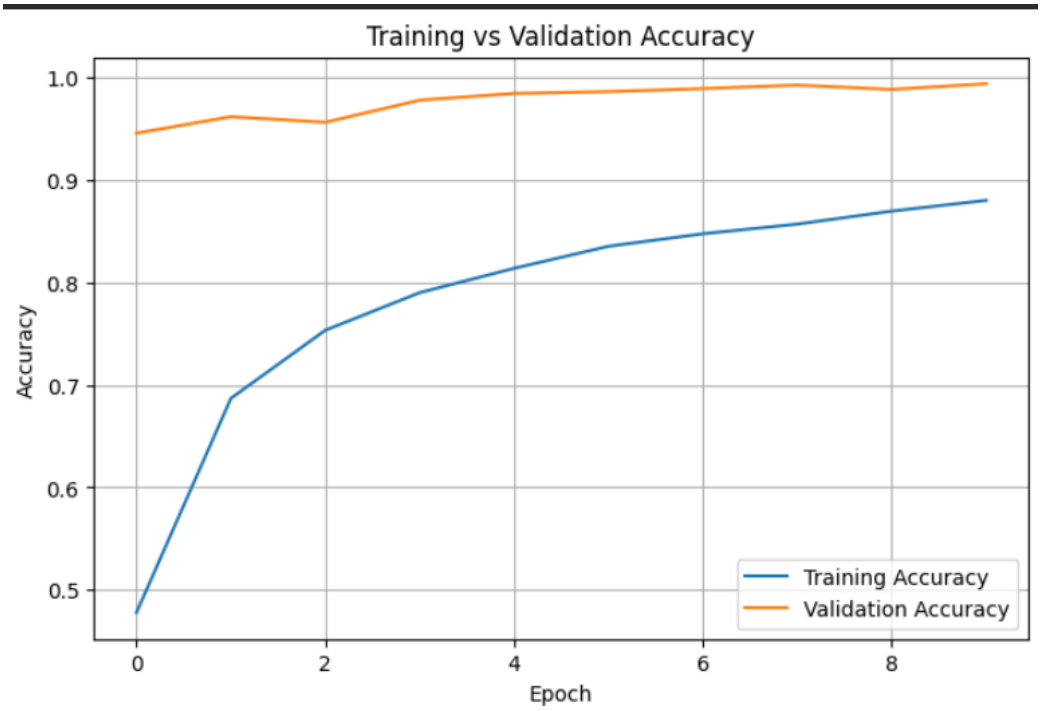


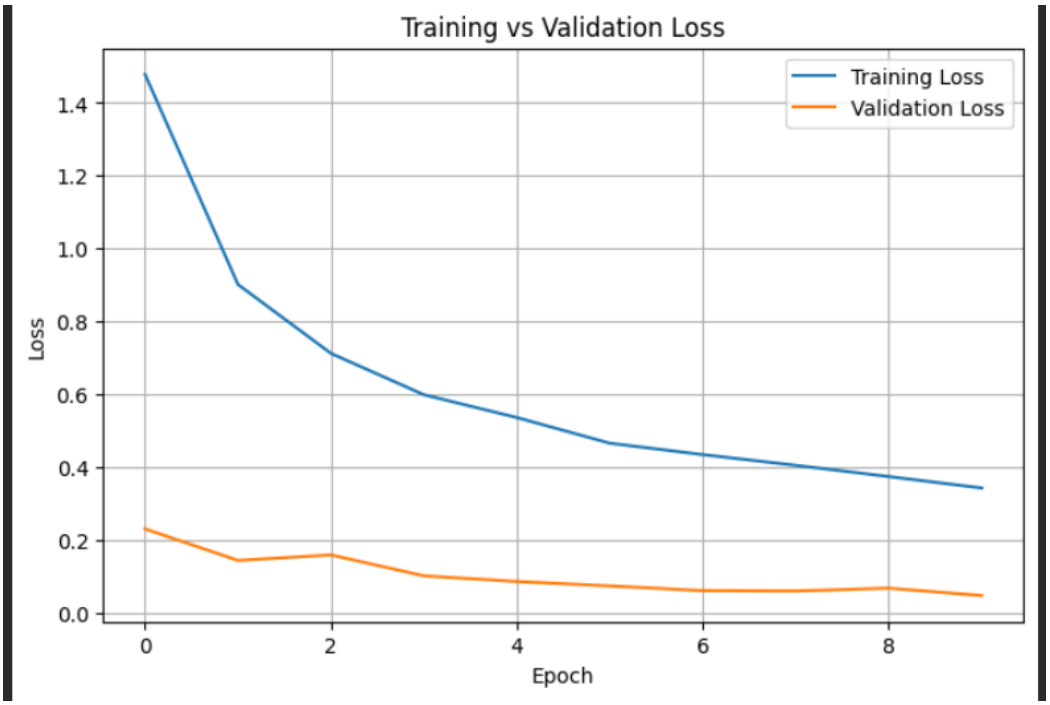
Model: "sequential_1"

Layer (type)	Output Shape	Param #
sequential (Sequential)	(None, 128, 128, 3)	0
rescaling (Rescaling)	(None, 128, 128, 3)	0
conv2d (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_1 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_2 (Conv2D)	(None, 28, 28, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten (Flatten)	(None, 25088)	0
dense (Dense)	(None, 128)	3,211,392
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

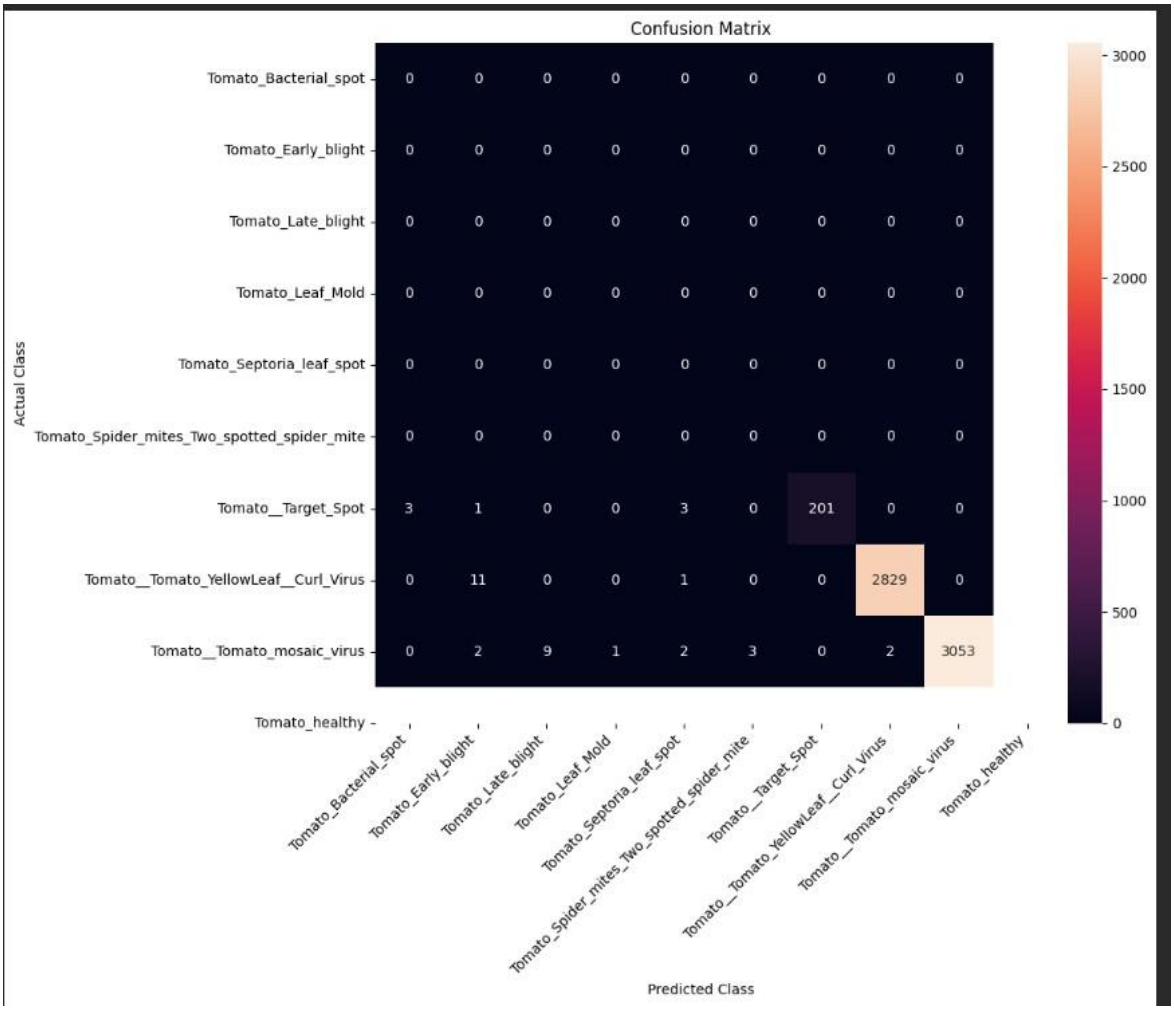
Total params: 3,305,930 (12.61 MB)
Trainable params: 3,305,930 (12.61 MB)
Non-trainable params: 0 (0.00 B)

```
Epoch 1/10
766/766 ━━━━━━━━━━━━━━━━━━━━ 31s 31ms/step - accuracy: 0.4780 - loss: 1.4783 - val_accuracy: 0.9456 - val_loss: 0.2315
Epoch 2/10
766/766 ━━━━━━━━━━━━━━━━━━━━ 36s 30ms/step - accuracy: 0.6869 - loss: 0.9019 - val_accuracy: 0.9616 - val_loss: 0.1450
Epoch 3/10
766/766 ━━━━━━━━━━━━━━━━━━━━ 40s 29ms/step - accuracy: 0.7533 - loss: 0.7129 - val_accuracy: 0.9562 - val_loss: 0.1601
Epoch 4/10
766/766 ━━━━━━━━━━━━━━━━━━━━ 41s 29ms/step - accuracy: 0.7900 - loss: 0.5994 - val_accuracy: 0.9778 - val_loss: 0.1027
Epoch 5/10
766/766 ━━━━━━━━━━━━━━━━━━━━ 24s 32ms/step - accuracy: 0.8137 - loss: 0.5369 - val_accuracy: 0.9843 - val_loss: 0.0873
Epoch 6/10
766/766 ━━━━━━━━━━━━━━━━━━━━ 23s 30ms/step - accuracy: 0.8352 - loss: 0.4665 - val_accuracy: 0.9861 - val_loss: 0.0751
Epoch 7/10
766/766 ━━━━━━━━━━━━━━━━━━━━ 23s 30ms/step - accuracy: 0.8474 - loss: 0.4352 - val_accuracy: 0.9891 - val_loss: 0.0621
Epoch 8/10
766/766 ━━━━━━━━━━━━━━━━━━━━ 41s 30ms/step - accuracy: 0.8569 - loss: 0.4058 - val_accuracy: 0.9925 - val_loss: 0.0611
Epoch 9/10
766/766 ━━━━━━━━━━━━━━━━━━━━ 41s 30ms/step - accuracy: 0.8695 - loss: 0.3750 - val_accuracy: 0.9882 - val_loss: 0.0690
Epoch 10/10
766/766 ━━━━━━━━━━━━━━━━━━━━ 24s 31ms/step - accuracy: 0.8800 - loss: 0.3436 - val_accuracy: 0.9938 - val_loss: 0.0486
```


Training vs Validation Accuracy
Accuracy
Epoch
Training Accuracy
Validation Accuracy
1.0
0.9
0.8
0.7
0.6
0.5
0
2
4
6
8


Training vs Validation Loss
Loss
Epoch
Training Loss
Validation Loss
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
0
2
4
6
8

```
192/192 ━━━━━━━━━━━━━━━━━━━━ 4s 19ms/step - accuracy: 0.9938 - loss: 0.0486
Validation Loss: 0.04859260842204094
Validation Accuracy: 0.9937918782234192
Validation Accuracy (%): 99.38
```



	precision	recall	f1-score	support
Tomato_Bacterial_spot	0.00	0.00	0.00	0
Tomato_Early_blight	0.00	0.00	0.00	0
Tomato_Late_blight	0.00	0.00	0.00	0
Tomato_Leaf_Mold	0.00	0.00	0.00	0
Tomato_Septoria_leaf_spot	0.00	0.00	0.00	0
Tomato_Spider_mites_Two_spotted_spider_mite	0.00	0.00	0.00	0
Tomato__Target_Spot	0.00	0.00	0.00	0
Tomato__Tomato_YellowLeaf__Curl_Virus	1.00	0.97	0.98	208
Tomato__Tomato_mosaic_virus	1.00	1.00	1.00	2841
Tomato_healthy	1.00	0.99	1.00	3072
accuracy			0.99	6121
macro avg	0.30	0.30	0.30	6121
weighted avg	1.00	0.99	1.00	6121